



Layout

In this chapter, you will learn how to:

- Create a custom header.
- Create a custom footer.
- Modify a standard template.
- Add a copyright section.
- Improve your website's responsiveness.

Default

An Odoo page combines cross-page and unique elements. Cross-page elements are the same on every page, while unique elements are only related to a specific page. By default, a page has two cross-page elements, the header and the footer, and a unique main element that contains the specific content of that page.

```
<div id="wrapwrap">
  <header/>
  <main>
    <div id="wrap" class="oe_structure">
      <!-- Page Content -->
    </div>
  </main>
  <footer/>
</div>
```

Any Odoo XML file starts with encoding specifications. After that, you must write your code inside an `<odoo>` tag.

```
<?xml version="1.0" encoding="utf-8" ?>
<odoo>
  ...
</odoo>
```

Note

configuring your IDE.

XPath

XPath (XML Path Language) is an expression language that enables you to navigate through elements and attributes in an XML document easily. XPath is used to extend standard Odoo templates.

A view is coded the following way.

```
<template id="..." inherit_id="..." name="...">
  <!-- Content -->
</template>
```

Attribute	Description
id	ID of the modified view
inherited_id	ID of the standard view (using the following pattern: <code>module.template</code>)
name	Human-readable name of the modified view

For each XPath, you modify two attributes: **expression** and **position**.

Example

/website_airproof/views/website_templates.xml

```
<template id="layout" inherit_id="website.layout" name="Wel
  <xpath expr="//header" position="before">
    <!-- Content -->
  </xpath>
</template>
```

This XPath adds a welcome message right before the page content.

one, your changes will take priority over any future Odoo update.

Note

- You should update your module every time you create a new template or record.
- *XML IDs* of inheriting views should use the same *ID* as the original record. It helps to find all inheritance at a glance. As final *XML IDs* are prefixed by the module that creates them, there is no overlap.

Expressions

XPath uses path expressions to select nodes in an XML document. Selectors are used inside the expression to target the right element. The most useful ones are listed below.

Descendent selectors

Description

/	Selects from the root node.
//	Selects nodes in the document from the current node that matches the selection no matter where they are.

Attribute selectors

Description

*	Selects any XML tag. * can be replaced by a specific tag if the selection needs to be more precise.
*[@id="id"]	Selects a specific ID.
*[hasclass("class")]	Selects a specific class.
*[@name="name"]	Selects a tag with a specific name.
*[@t-call="t-call"]	Selects a specific t-call.

Position

Position	Description
replace	Replaces the targeted node with the XPath content.
inside	Adds the XPath content inside the targeted node.
before	Adds the XPath content before the targeted node.
after	Adds the XPath content after the targeted node.
attributes	Adds the XPath content inside an attribute.

Example

This XPath removes the first element with a `.breadcrumb` class.

```
<xpath expr="//*[hasclass('breadcrumb')]" position="replace"/>
```

This XPath adds an extra `<li>` element after the last child of the `<ul>` element.

```
<xpath expr="//ul" position="inside">
  <li>Last element of the list</li>
</xpath>
```

This XPath adds a `<div>` before the `<nav>` that is a direct child of the `<header>`.

```
<xpath expr="//header/nav" position="before">
  <div>Some content before the header</div>
</xpath>
```

This XPath removes `x_airproof_header` in the class attribute of the header. In this case, you don't need to use the `separator` attribute.

```
<xpath expr="//header" position="attributes">
  <attribute name="class" remove="x_airproof_header" />
</xpath>
```

```
<xpath expr="//header" position="attributes">
  <attribute name="class" add="x_airproof_header" separator=" " />
</xpath>
```

This XPath moves the element with `.o_footer_scrolltop_wrapper` class before the element with the `footer` ID attribute.

```
<xpath expr="//div[@id='footer']" position="before">
  <xpath expr="//div[@id='o_footer_scrolltop_wrapper']" position="move" />
</xpath>
```

Tip

Using `move` directives inside an other XPath forces you to use only this kind of directives.

Example

Good example:

```
<xpath expr="//*[hasclass('o_wsale_products_main_row')]" position="before">
  <xpath expr="//t[@t-if='opt_wsale_categories_top']" position="move" />
</xpath>
<xpath expr="//*[hasclass('o_wsale_products_main_row')]" position="before">
  <div><!-- Content --></div>
</xpath>
```

Bad example:

```
<xpath expr="//*[hasclass('o_wsale_products_main_row')]" position="before">
  <xpath expr="//t[@t-if='opt_wsale_categories_top']" position="move" />
  <div><!-- Content --></div>
</xpath>
```

QWeb

QWeb is the primary templating engine used by Odoo. It is an XML templating engine mainly used to generate HTML fragments and pages.

`t-call` directives can receive parameters:

```
<!-- Old way -->
<t t-call="portal.user_dropdown">
  <t t-set="_icon" t-value="True" />
  <t t-set="_item_class" t-value="dropdown" />
</t>

<t t-call="website.layout">
  <t t-set="additional_title">My page title</t>
</t>

<!-- New way (Parametric) -->
<t t-call="portal.user_dropdown"
  _icon="true"
  _item_class.f="dropdown" />

<t t-call="website.layout"
  additional_title.translate="My page title" />
```

Attribute (examples)	Description
<code>_icon="true"</code>	Pass the raw value (here a boolean set to <code>true</code>)
<code>_item_class.f="dropdown"</code>	Pass value as a string
<code>additional_title.translate="My page title"</code>	Pass value as a string

See also

[QWeb templates documentation.](#)

Declaration

First, create a record to declare the field. This field has to be linked to an existing model.

/website_airproof/data/fields.xml

```
<record id="x_post_category" model="ir.model.fields">
  <field name="name">x_post_category</field>
  <field name="field_description">...</field>
  <field name="ttype">html</field>
  <field name="state">>manual</field>
  <field name="index">0</field>
  <field name="model_id" ref="website_blog.model_blog_post"
</record>
```

Note

Fields creation is also possible (and recommended) through [a model using Python](#).

Back-end

Add the field to the relevant view through an XPath. Therefore, the user can see the field in the interface and fill it afterwards.

/website_airproof/views/backend/website_blog_views.xml

```
<record id="view_blog_post_form_category" model="ir.ui.view">
  <field name="name">view_blog_post_form_category</field>
  <field name="model">blog.post</field>
  <field name="inherit_id" ref="website_blog.view_blog_post"
  <field name="arch" type="xml">
    <xpath expr="//field[@name='blog_id']" position="before">
      <field name="x_post_category" string="..." placeholder="..."
    </xpath>
  </field>
</record>
```

Front-end

```
<h1 t-field="blog_post.x_post_category" />
```

Background

You can define a color or an image as the background of your website.

Colors

/website_airproof/static/src/scss/primary_variables.scss

```
$o-color-palettes: map-merge($o-color-palettes,  
  (  
    'airproof': (  
      'o-cc1-bg': 'o-color-5',  
      'o-cc5-bg': 'o-color-1',  
    ),  
  )  
);
```

Image/pattern

/website_airproof/static/src/scss/primary_variables.scss

```
$o-website-values-palettes: (  
  (  
    'body-image': '/website_airproof/static/src/img/backg  
    'body-image-type': 'image' or 'pattern'  
  )  
);
```

Header

By default, the header contains two distinct templates (desktop and mobile) which display the main navigation, the company's logo and other optional elements (call-to-action, language selector, etc). Depending on the situation, choose between enabling/disabling existing elements with a standard template or building a brand new custom template.

Desktop template

Enable one of the header default templates.

Important

Don't forget that you may need to disable the active header template first.

Example

/website_aiproof/data/presets.xml

```
<record id="website.template_header_default" model="ir.ui.v
  <field name="active" eval="False" />
</record>
```

Explicitly set the desired template in the `primary_variables.scss` file.

/website_airproof/static/src/scss/primary_variables.scss

```
$o-website-values-palettes: (
  (
    'header-template': 'stretch',
  ),
);
```

/website_aiproof/data/presets.xml

```
<record id="website.template_header_stretch" model="ir.ui.v
  <field name="active" eval="True" />
</record>
```

Mobile template

Each header template comes with the `template_header_mobile` template ensuring a seamless user experience accross every devices.

Custom

Create your own template and add it to the list.

Important

Don't forget that you may need to disable the active header template first before enabling the custom one.

Option

Use the following code to add an option for your new custom header on the Website Builder.

/website_airproof/static/src/website_builder/header_template_option.xml

```
<t t-name="website_airproof.HeaderTemplateOption" t-inherit="website_builder.HeaderTemplateOption" t-inherit-mode="extension">
  <xpath expr="//BuilderRow[@label.translate='Template']//t>
    <BuilderSelectedItem
      id="'header_airproof_opt'"
      title.translate="Airproof"
      actionParam="[
        {
          action: 'websiteConfig',
          actionParam: {
            views: ['website_airproof.header'],
            vars: { 'header-template': 'airproof' },
            checkVars: false,
          },
        },
      ]">
      <Img src="'/website_airproof/static/src/img/wbuild'
    </BuilderSelectedItem>
  </xpath>
</t>
```

vars	The name given to the variable (same as used into <code>primary_variables.scss</code>)
checkVars	Determine if <code>vars</code> are compared to set the option status.
src (in <code>Img</code>)	The thumbnail of the custom template shown in the templates selection on the Website Builder

Now you have to explicitly define that you want to use your custom template in the Odoo SASS variables.

```
/website_airproof/static/src/scss/primary_variables.scss
```

```
$o-website-values-palettes: (
  (
    'header-template': 'airproof',
  ),
);
```

Template

```
/website_airproof/views/website_templates.xml
```

```
<template id="header" inherit_id="website.layout" name="Airp
  <xpath expr="//header//nav" position="replace">
    <!-- Static Content -->
    <!-- Components -->
    <!-- Editable areas -->
  </xpath>
</template>
```

Don't forget to adapt the `template_header_mobile` accordingly to keep consistency between desktop and mobile:

```
website_airproof/views/website_templates.xml
```

```
<template id="template_header_mobile" inherit_id="website.t
  <!-- Xpaths -->
```

Components

In your custom header, you can call several sub-templates using the `t-call` directive from QWeb:

Logo

```
<t t-call="website.placeholder_header_brand"
  _link_class.f="..." />
```

Important

Don't forget to **create a record of the website logo** logo in the database.

Menu

```
<t t-foreach="website.menu_id.child_id" t-as="submenu">
  <t t-call="website.submenu"
    item_class.f="nav-item"
    link_class.f="nav-link" />
</t>
```

Sign in

```
<t t-call="portal.placeholder_user_sign_in"
  _item_class.f="nav-item"
  _link_class.f="nav-link" />
```

User dropdown

```
<t t-call="portal.user_dropdown"
  _user_name="true"
  _icon="false"
  _avatar="false"
  _item_class.f="nav-item dropdown">
```

Language selector

```
<t t-call="website.placeholder_header_language_selector"
  _div_classes.f="..." />
```

Call to action

```
<t t-call="website.placeholder_header_call_to_action"
  _div_classes.f="..." />
```

Navbar toggler

```
<t t-call="website.navbar_toggler"
  _toggler_class.f="..." />
```

See also

You can add a **header overlay** to position your header over the content of your page. It has to be done on each page individually.

Footer

By default, the footer contains a section with some static content. You can easily add new elements or create your own template.

Standard

Enable one of the default footer templates. Don't forget that you may need to disable the active footer template first.

Important

Don't forget that you may need to disable the active footer template first.

Example

```
/website_aiproof/data/presets.xml
```

```
/website_airproof/static/src/scss/primary_variables.scss
```

```
$o-website-values-palettes: (
  (
    'footer-template': 'Links',
  ),
);
```

```
/website_airproof/data/presets.xml
```

```
<record id="website.template_footer_links" model="ir.ui.view">
  <field name="active" eval="True" />
</record>
```

Custom

Create your own template and add it to the list. Don't forget that you may need to disable the active footer template first.

Option

```
/website_airproof/static/src/website_builder/footer_option_plugin.js
```

```
import { Plugin } from "@html_editor/plugin";
import { registry } from "@web/core/registry";
import { _t } from "@web/core/l10n/translation";
import { FooterTemplateChoice } from "@website/builder/plugin";

export class AirproofFooterOptionPlugin extends Plugin {
  static id = "airproofFooterOption";
  resources = {
    footer_templates_providers: () => [
      {
        key: "airproof",
        Component: FooterTemplateChoice,
        props: {
```

```
        },
    },
],
};
}

registry.category("website-plugins").add(AirproofFooterOption)
```

Property	Description
title	Display title of the template
view	Template that is enabled.
varName	Value used in <code>primary_variables.scss</code> under <code>footer-template</code> .
imgSrc	The thumbnail of the custom template shown in the templates selection on the Website Builder

Declaration

/website_airproof/static/src/scss/primary_variables.scss

```
$o-website-values-palettes: (
  (
    'footer-template': 'airproof',
  ),
);
```

Template

/website_airproof/views/website_templates.xml

```
<template id="footer" inherit_id="website.layout" name="Airproof Footer"
  <xpath expr="//div[@id='footer']" position="replace">
    <div id="footer" class="oe_structure oe_structure_sol
      <!-- Content -->
    </div>
```

Copyright

There is only one template available at the moment for the copyright bar.

To replace the content or modify its structure, you can add your own code to the following XPath.

/website_airproof/views/website_templates.xml

```
<template id="copyright" inherit_id="website.layout">
  <xpath expr="//div[class='o_footer_copyright']" position="replace">
    <div class="o_footer_copyright" data-name="Copyright">
      <!-- Content -->
    </div>
  </xpath>
</template>
```

Drop zone

Instead of defining the complete layout of a page, you can create building blocks (snippets) and let the user choose where to drag and drop them, creating the page layout on their own. We call this *modular design*.

You can define an empty area that the user can fill with snippets.

```
<div id="oe_structure_layout_01" class="oe_structure" />
```

Class	Description
oe_structure	Define a drag-and-drop area for the user.
oe_structure_solo	Only one snippet can be dropped in this area.
oe_structure_not_nearest	If a building block is dropped outside of a Drop zone having this class, the block will be moved in the nearest Drop Zone.

You can also populate an existing drop zone with your content.


```
<!-- Content -->
</div>
</xpath>
</template>
```

Responsive

Odoo in general relies on the Bootstrap framework which eases the responsiveness of your website on desktop and mobile. You can mainly take action on 3 aspects:

1. Automatic computed font sizes depending on the device
2. Column sizes on desktop (the columns are automatically stacked on mobile)
3. Visibility conditions (Show/Hide something on desktop/mobile)

See also

- [Bootstrap documentation on display property](#)

Font sizes

In Bootstrap 5, responsive font sizes are enabled by default, allowing text to scale more naturally across device and viewport sizes (relying on the `$enable-rfs` variable).

See also

- [Bootstrap documentation about responsive font sizes](#)

Column sizes

Bootstrap uses a grid made of rows and columns to layout a page. Thanks to this structure, columns can be sized differently on mobile and desktop. In this version, the Website Builder allows to set mobile sizes (`col-12` for example) and desktop ones (`col-lg-4` for example) but not the medium breakpoints (`col-md-4` for example).

Warning

The medium sizes can be set but the end-user is not able to edit them within the Website Builder.

• [Bootstrap documentation about the grid](#)

Visibility conditions

In the Odoo Website Builder, entire sections or specific columns can be hidden on mobile or desktop.

This functionality leverages Bootstrap along with Odoo-specific classes:

- `o_snippet_mobile_invisible`
- `o_snippet_desktop_invisible`

Hide a section on desktop:

```
<section
  class="s_text_block o_cc o_cc1 o_colored_level pt16 pb16 d-lg-none o_snippet_desktop_invisible"
  data-snippet="s_text_block"
  data-name="Text">
  <!-- Content -->
</section>
```

Hide a column on mobile:

```
<section
  class="s_text_block o_cc o_cc1 o_colored_level pt16 pb16"
  data-snippet="s_text_block"
  data-name="Text">
  <div class="container s_allow_columns">
    <div class="row">
      <div class="col-12 col-lg-6 d-none d-lg-block o_snippet_mobile_invisible">
        Column 1
      </div>
      <div class="col-12 col-lg-6">
        Column 2
      </div>
    </div>
  </div>
</section>
```

using visibility conditions option.

o_snippet_desktop_invisible It tells the Website Builder that the element is hidden **on desktop and** is using visibility conditions option.

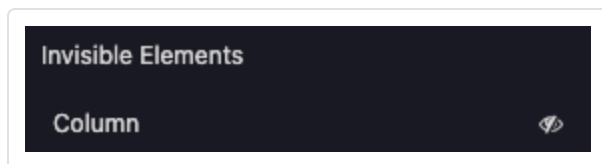
d-none Hide the element in every situations.

d-lg-block Show the element from the “large” breakpoint (on desktop).

Important

o_snippet_mobile_invisible / **o_snippet_desktop_invisible** classes have to be specified to keep

the visibility conditions option functional. Even if an element is hidden on desktop, the Website Builder displays a list of these elements allowing the end-user to force show the element and edit it without switching between mobile and desktop mode.



Get Help

Contact Support

Ask the Odoo Community